

Dynamic Resource Allocation in Cloud Distribution Systems

Aim to combine Deep-Learning with Online Convex Optimization to lower operational costs while maintaining SLA Guarantees under non-stationary traffic spikes

Thesis: A hybrid dynamic resource allocation framework combining predictive deep-learning schedulers for baseline demand with Online Convex Optimization for real-time drift correction achieves lower operational costs while maintaining SLA Guarantees under non-stationary traffic spikes

Goal Statement: I'm interested if cloud services can be optimized using two techniques, firstly using OCO Algorithms to enhance delivery speeds and minimize overhead latency. The other technique is using deep learning schedulers to potentially improve the dynamic resource allocation on a predictive model using usage data. Lastly, testing the combination of the two and attempting to create a improved backend for cloud servers and identifying the positives and negatives of this approach.

Tech Stack:

- ML / Optimization PyTorch, NumPy, SciPy
- Cloud & Containers: Docker, FastAPI, Prometheus, Grafana
- Traffic Testing: Custom python async traffic generator

Project One: Predictive-Dynamic Micro-Load Balancer

Introduction: This starter project isolates the core hybrid architecture, a predictive model handling baseline demand paired with a real time reactive feedback loop. It avoids cloud infrastructure bills while merging concepts from machine learning, cloud API operations, and dynamic optimization.

Time Frame: 3-5 Weeks

Phase 1: Workload Generator & Local Cluster

Goal: set up a minimal, inspectable backend environment using Docker and Python

Hardware: Local Docker Desktop

Service Architecture: Deploy fastAPI app running behind 1 to 10 container instances

Traffic Generator: Use a python script to simulate incoming request per second, combine with a predictable sine wave pattern (to represent daily cyclic traffic) with random noise (variation within traffic) and sudden spikes (unpredictable surges).

This phase will prepare data for the predictive scheduler that will be used during the next phase and will setup the VM to contain our scripts and data, allowing for accurate measurements of our metrics.

Phase 2: Predictive Baseline Scheduler

Introduction: Build predictive engine to allocate container capacity in advance based on historical usage pattern learning.

Model Architecture: Build a light PyTorch LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) Model

Input Data: Time series sequence of past RPS and CPU utilization metrics (Metrics collected during phase 1) following $t_{-n} \dots t_0$

Output: Predicted demand for the next time window ($t_{+1} \dots t_{+10}$), mapped directly to target container scale count

$$\text{baseline Containers} = \left\lceil \frac{\text{predicted RPS}}{\text{Capacity per Container}} \right\rceil$$

Phase 3: Real-Time Drift Correction Engine (OCO)

Implementation of Online Convex Optimization to correct the baseline scheduler's prediction errors instantly.

Algorithm: Implement OGD (Online Gradient Descent) to adjust resource allocations dynamically:

$$x_{t+1} = \Pi_x(x_t - \eta \nabla f_t(x_t))$$

Loss Function $f_t(x_t)$: Define a convex cost function balancing SLA penalty and resource overhead:

$$f_t(x_t) = c_{sla} \cdot \max(0, Latency_t - SLA_{tar}) + c_{resource} \cdot x_t$$

Where x_t represents added capacity and η is the learning rate.

Execution: Run the OGD loop every second. If actual traffic exceeds the LSTM prediction OGD quickly scales up resources before SLA violations compound.

Phase 4: Hybrid integration & metrics Benchmarking

Combine both modules into a unified controller and run comparative experiments against baseline strategies

Metric	Reactive-Only	Predictive-Only	OCO-Only	Hybrid Framework
SLA Violations (%)				
Resource Waste (Over-Provisioning)				
Response Latency				